A REPORT ON

A ROBOT-AGNOSTIC FRAMEWORK FOR LANGUAGE-CONDITIONED TASK PLANNING USING CAPABILITY-AWARE LARGE LANGUAGE MODELS

BY

Sana Jaya Krishna

ID No.: 2024AA05783

AT

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

VIDYA VIHAR, PILANI, RAJASTHAN – 333031

[July, 2026]

A REPORT ON

A ROBOT-AGNOSTIC FRAMEWORK FOR LANGUAGE-CONDITIONED TASK PLANNING USING CAPABILITY-AWARE LARGE LANGUAGE MODELS

BY

Sana Jaya Krishna

ID No.: 2024AA05783

Prepared in fulfilment of the

Degree Programme: M.Tech. Artificial Intelligence and Machine Learning
WILP Dissertation Course (Course No.: AIMLCZG628T)

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

[July, 2026]

Abstract:

Modern robots exhibit heterogeneous embodiments with varying mobility, manipulation, perception, and operational capabilities. A logically correct plan generated by a Large Language Model (LLM) may be infeasible if the assigned robot lacks the required capabilities. This dissertation presents a robot-agnostic, capability-aware framework for language-conditioned robotic task planning. The proposed approach derives structured capability representations from robot description files (URDF/Xacro) and provides them as context to an LLM planner alongside world-state representations and a standardized skills library. The framework is robot-agnostic in architecture but robot-aware during planning. A deterministic software-based evaluation module assesses generated plans across six dimensions: goal achievement, action validity, object and location validity, capability compliance, logical ordering, and plan feasibility — without relying on a secondary LLM as a judge. A custom dataset of approximately 500 natural-language tasks across 11 diverse environments was developed to benchmark multiple lightweight open-source LLMs under identical experimental conditions, enabling controlled comparison of planning performance and providing a baseline for subsequent parameter-efficient fine-tuning (PEFT) experiments. The framework establishes a foundational architecture and evaluation methodology for capability-aware robotic task planning.

Contents

List of Abbreviations and Acronyms

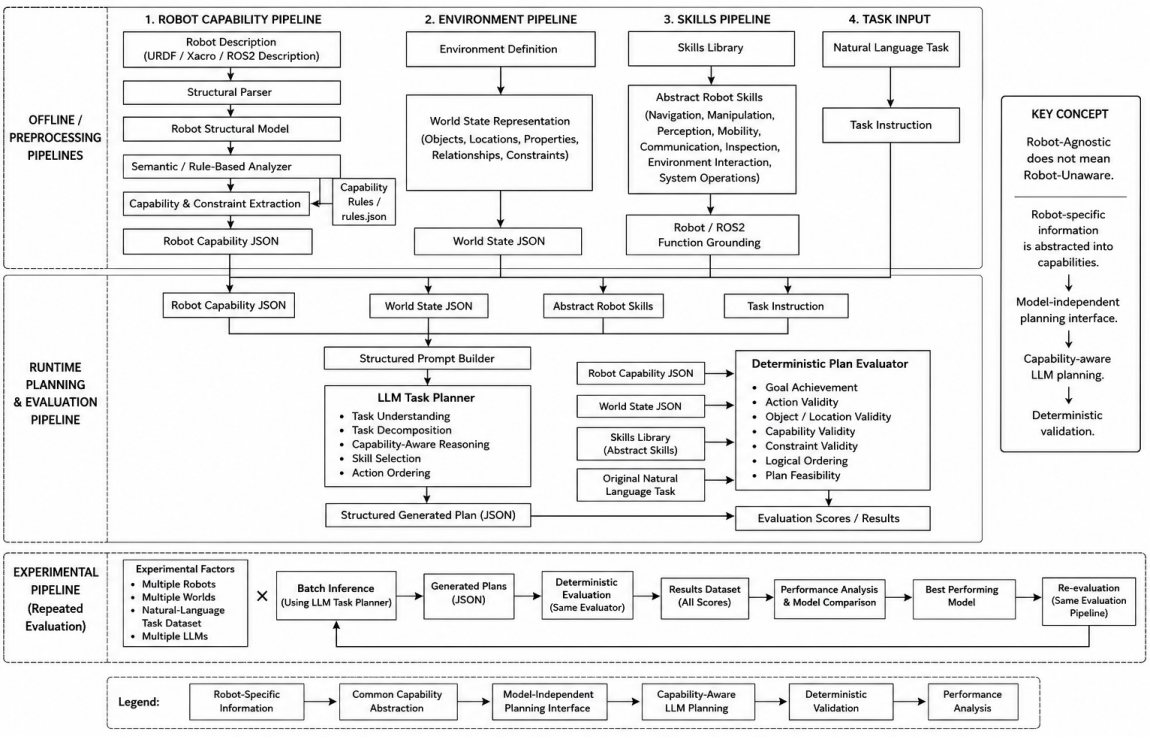
AI	Artificial Intelligence
EMOS	Embodiment-aware Multi-rObot System
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
ML	Machine Learning
MoveIt2	Motion Planning Framework for ROS2
Nav2	Navigation2 Stack for ROS2
PEFT	Parameter-Efficient Fine-Tuning
ROS2	Robot Operating System 2
TAMP	Task and Motion Planning
URDF	Unified Robot Description Format
WILP	Work Integrated Learning Programme
Xacro	XML Macros (robot description extension)

1 Introduction and Research Motivation

Robotics is evolving from pre-programmed systems toward intelligent agents capable of understanding natural-language instructions and operating across diverse environments. Large Language Models (LLMs) provide strong reasoning and task-decomposition capabilities, and approaches such as SayCan [1], ProgPrompt [2], and EMOS [3] have demonstrated their potential for robotic task planning. However, existing approaches often depend on predefined action spaces, robot-specific configurations, or limited representations of robot capabilities.

A key challenge is that **robots have different embodiments and capabilities**. A plan that is logically correct may still be infeasible if the selected robot lacks the required mobility, manipulation, perception, payload, or software skills. This creates a gap between **LLM-generated plans and physically executable robotic plans**.

This dissertation addresses this gap through a **robot-agnostic, capability-aware framework for language-conditioned task planning**. The proposed approach can be summarized as:



Overall Architecture of the Proposed Robot-Agnostic Capability-Aware Language-Conditioned

Task Planning Framework

Robot Description → Capability Representation → LLM Planning → Capability-Aware Plan

Robot descriptions such as URDF [5] are transformed into structured capabilities and combined with world information and available robotic skills to ground LLM planning. The framework is therefore **robot-agnostic in architecture but robot-aware during planning**.

The work establishes a foundational architecture, capability representation, planning pipeline, dataset, and evaluation methodology. It provides a basis for future research involving real-world execution, dynamic capability learning, closed-loop planning, multi-robot systems, and broader cross-robot generalization.

2 Literature Review and Research Gap

Language-conditioned robotic planning has gained significant attention with the development of Large Language Models [4]. Existing research demonstrates that LLMs can interpret high-level instructions and decompose them into sequences of robotic actions.

SayCan [1] combines LLM-generated action probabilities with learned affordance functions to select actions that are both relevant to the instruction and feasible for the robot. While effective, its planning process relies on a predefined skill set and robot-specific affordance models.

ProgPrompt [2] uses programmatic prompting to generate structured robot task plans. It improves the organization and interpretability of LLM-generated plans but provides limited explicit reasoning about the physical capabilities of different robot embodiments.

EMOS [3] extends embodiment-aware reasoning by providing structured information about robots to LLM-based agents, particularly for heterogeneous and multi-robot environments. This demonstrates the importance of representing embodiment during planning,

but leaves opportunities for more explicit capability extraction, constraint representation, and executable skill grounding. These limitations were identified as the primary motivation for the proposed dissertation framework.

2.1 Research Gap

Existing approaches demonstrate the effectiveness of LLMs for robotic reasoning, but a gap remains between **language-level planning and embodiment-level feasibility** [4]. In particular, there is a need for a framework that:

- represents heterogeneous robots through a common capability abstraction,
- derives capabilities systematically from robot descriptions,
- grounds generated actions against available robotic skills and environmental constraints, and
- evaluates generated plans independently of another LLM.

This dissertation addresses this gap by introducing a **robot-agnostic capability representation and deterministic evaluation layer around LLM-based task planning**, allowing different robots, environments, tasks, and language models to be evaluated within a common framework.

3 Problem Definition and Objectives

3.1 Problem Definition

LLMs can generate logically meaningful task plans from natural-language instructions, but they do not inherently guarantee that those plans are feasible for a specific robot. Differences in mobility, manipulation, perception, sensors, payload, reach, and available software skills can make the same task executable by one robot and infeasible for another.

The problem addressed in this dissertation is therefore:

> **How can natural-language task planning be made robot-agnostic while ensuring that generated plans remain grounded in the capabilities and constraints of the selected robot and environment?**

The proposed framework addresses this by separating **robot capability representation**, **environment representation**, **available skills**, and **LLM-based reasoning**, allowing the planner to operate using structured information rather than relying solely on the LLM's internal knowledge.

3.2 Research Objectives

The primary objectives of this dissertation are:

- Develop a structured and robot-independent representation of robotic capabilities.
- Extract relevant capabilities from robot descriptions such as URDF/Xacro [5].
- Represent executable robotic operations through a common skills library.
- Provide robot capabilities, world state, skills, and task instructions as structured context to an LLM.
- Generate structured, capability-aware task plans across different robots and environments.
- Develop a deterministic software-based evaluator to assess plan validity and feasibility without relying on another LLM.
- Benchmark multiple language models and study their suitability for capability-aware robotic task planning.

These objectives extend the initial dissertation scope of capability extraction, LLM integration, feasibility-aware planning, and cross-robot evaluation into a complete experimental framework.

4 Proposed Framework and System Architecture

The proposed system follows a modular architecture that separates **robot representation, environment representation, task reasoning, skill grounding, and plan evaluation**. This separation enables the same planning pipeline to operate across different robot embodiments and environments.

The overall workflow is:

$$\textit{Robot Description} \rightarrow \textit{Capability Extraction} \rightarrow \textit{Capability Representation}$$
$$\rightarrow \textit{LLM Planner} \rightarrow \textit{Generated Plan} \rightarrow \textit{Deterministic Evaluation}$$

The framework consists of the following major components:

- **Capability Extractor:** Parses robot descriptions such as URDF/Xacro [5] and derives structured information about mobility, manipulation, sensors, joints, reach, payload-related constraints, and other relevant capabilities.
- **Capability Representation:** Converts robot-specific information into a standardized JSON representation that can be consumed independently of the robot platform.
- **World State Representation:** Describes the environment, including locations, objects, properties, relationships, and environmental constraints relevant to planning.
- **Skills Library:** Defines the abstract robotic operations available to the planner and provides a bridge between high-level actions and underlying robotic functions.
- **LLM Planner:** Receives the natural-language task together with robot capabilities, world state, and available skills to generate a structured sequence of actions.
- **Evaluation Module:** Deterministically evaluates the generated plan for factors such as goal achievement, action and object validity, capability constraints, and logical ordering.

  The framework will be evaluated using the following quantitative metrics:

- **Task Success Rate (TSR):** $\text{TSR} = \left(\frac{\text{Successful tasks}}{\text{Total tasks}}\right) \times 100$
- **Plan Feasibility Rate:** $\text{Feasibility} = \left(\frac{\text{Valid plans}}{\text{Total generated plans}}\right) \times 100$
  (A plan is valid if all actions satisfy robot capability constraints)
- **Execution Success Rate:** $\text{Execution Success Rate} = \frac{\text{Successfully executed plans}}{\text{Total executed plans}}$
- **Plan Efficiency:** $\text{Efficiency} = \frac{\text{Optimal steps}}{\text{Generated steps}}$
- **Generalization Score:** Average performance across different robot types (robot-agnostic capability)
- **Instruction Understanding Accuracy:** $\text{Accuracy} = \frac{\text{Correct interpretations}}{\text{Total instructions}}$

These metrics combine both logical validation (constraint satisfaction) and quantitative evaluation.

This architecture extends the capability-aware planning concept, where robot capability representation was proposed as an intermediate layer between LLM reasoning and robot execution.

The modular design also allows individual components—such as the LLM, robot description, environment, skills library, or evaluator—to be modified independently, making the framework suitable for systematic experimentation and future extension.

The plan structure within this framework draws inspiration from **NavTopo** [11], a topological navigation approach that organizes robot navigation as a sequence of waypoint sub-tasks rather than a monolithic path. Adopting a similar philosophy, task plans in the proposed framework are structured as **ordered, checkpointed sub-task sequences**. When execution feedback or replanning is triggered, only the affected checkpoint needs to be regenerated rather than the entire plan. This selective replanning strategy reduces LLM inference token cost significantly, since the context provided to the model is scoped to the failed sub-task and its local constraints rather than the complete task.

5 Capability Extraction and Robot Representation

A core component of the proposed framework is the **Capability Extractor**, which converts robot-specific descriptions into a standardized representation that can be understood by the planning system.

Robot description files such as **URDF/Xacro** [5] contain structural information including links, joints, actuators, sensors, and kinematic relationships. However, providing these files directly to an LLM introduces unnecessary complexity and does not explicitly describe what the robot can perform. Therefore, the framework introduces an intermediate capability abstraction.

The extraction pipeline follows:

$$\textit{URDF/Xacro} \rightarrow \textit{Structural Parsing} \rightarrow \textit{Semantic Analysis} \rightarrow$$
$$\textit{Capability Identification} \rightarrow \textit{Constraint Generation} \rightarrow \textit{Capability JSON}$$

A rule-based mechanism interprets structural properties and maps them to planning-relevant capabilities such as:

- mobility and locomotion,
- manipulation and end-effector availability,
- perception and sensing,
- communication and interaction, and
- kinematic and operational constraints.

The resulting **Capability JSON** acts as a standardized robot profile supplied to the planner. This allows different robot platforms to be represented through the same schema while retaining their individual capabilities and limitations.

The approach extends the initial concept of a structured *robot resume* proposed during the mid-semester phase into a deterministic capability extraction and representation pipeline.

This abstraction is fundamental to the robot-agnostic design: **the planner does not need to understand each robot's URDF directly; it reasons over a common representation of what the robot can and cannot do.**

6 Skills Library and Action Grounding

The **Skills Library** provides a standardized interface between high-level LLM-generated plans and executable robotic functions. While the capability representation describes **what a robot can potentially perform**, the skills library defines **which actions are available to the planning system and how they can be executed**.

Each skill is represented using a structured schema containing information such as the abstract skill name, required parameters, preconditions, capability requirements, constraints, and corresponding robotic implementation.

The library covers major robotic operations including:

- Navigation and mobility
- Manipulation and object interaction
- Perception and sensing
- Inspection and environment interaction
- Communication and system operations

For example, a high-level action such as `pick_object` can be associated with manipulation requirements and mapped to an appropriate ROS2 / MoveIt2 [7] implementation. Similarly, navigation actions can be grounded to corresponding Nav2 [8] navigation interfaces.

The resulting planning hierarchy is:

Natural-Language Task $\rightarrow$ *Abstract Skills* $\rightarrow$ *Capability Validation* $\rightarrow$ *Robot Functions*

By separating abstract skills from robot-specific implementations, the framework avoids directly coupling the LLM to individual ROS2 [6] functions. This enables a common planning vocabulary across heterogeneous robots while allowing the underlying implementation to vary according to the selected robotic platform.

Together, the **Capability Representation and Skills Library** form the grounding layer that connects language-based reasoning with executable robotic systems.

7 LLM-Based Task Planning

The **LLM Planner** acts as the reasoning component of the framework. Its objective is to convert a natural-language task into a structured sequence of robotic actions while considering the selected robot, environment, and available skills.

The planner receives four primary inputs:

$$\textit{Natural-Language Task} + \textit{Robot Capabilities} + \textit{World State} + \textit{Skills Library} \rightarrow \textit{Generated Plan}$$

Rather than allowing the LLM to generate unrestricted textual instructions, the framework uses a structured prompt and predefined output schema. The model is expected to decompose the requested task into ordered actions, identify relevant objects and locations, select appropriate skills, and respect the capabilities and constraints provided in the context.

For example, a task such as *"Collect the package and deliver it to the reception desk"* may be decomposed into:

$$\textit{Navigate} \rightarrow \textit{Detect Object} \rightarrow \textit{Pick Object} \rightarrow \textit{Navigate} \rightarrow \textit{Place Object}$$

However, the generated sequence depends on whether the robot possesses the required navigation, perception, and manipulation capabilities and whether the referenced objects and locations exist in the supplied world state.

The framework is also **model-agnostic**: the planning interface is designed so that different LLMs can be evaluated using the same task, robot capability representation, world state, skills, output structure, and evaluation methodology. This enables systematic comparison of models based on their ability to generate valid and capability-aware robotic plans rather than relying solely on general language-generation performance.

7.1 Inference Strategy: Zero-Shot and Prompt Engineering

The LLM planner in the current work is evaluated under a **zero-shot inference** regime: no task-specific training examples are provided to the model at inference time. The model is required to generate a valid, capability-aware plan based solely on the structured context — robot capabilities, world state, skills, and task description — supplied in a single prompt.

To improve zero-shot performance, the framework employs **structured prompt engineering**. The prompt is designed with a clear role definition, explicit output format specification (JSON schema), constraint declarations derived from the capability representation, and examples of valid action names from the skills library. This structured prompting strategy reduces hallucination, enforces schema compliance, and guides the model toward grounded action selection without requiring fine-tuning.

The current experimental scope is limited to a **small number of inference runs** due to computational resource constraints. The available compute restricts the number of robot-environment-task combinations that can be fully evaluated. Despite this, the framework design supports arbitrary scale-out: once sufficient inference results are available, **parameter-efficient fine-tuning (PEFT)** [9] such as LoRA can be applied to the best-performing base model to adapt it specifically for capability-aware task planning. Alternatively, the dataset of generated plans and evaluation scores could serve as supervision for training a **dedicated lightweight neural network** with a task-planning-specific architecture, reducing dependence on general-purpose LLMs for future deployment.

8 Dataset and Experimental Design

To systematically evaluate the proposed framework, a custom task-planning dataset was developed across **11 diverse environments**, including Kitchen, Apartment, Hospital, Restaurant, Delivery Warehouse, Factory, Chemistry Laboratory, Shopping Complex, Playground, College, and Hotel.

The dataset contains approximately **500 natural-language robotic tasks** with varying levels of complexity, ranging from simple actions to navigation-manipulation tasks, conditional tasks, constraint-aware tasks, and long-horizon plans. The environments are represented using structured world-state descriptions containing relevant objects, locations, properties, and relationships.

For the experimental study, selected humanoid robot configurations are evaluated across these environments using multiple lightweight open-source LLMs. Each experiment follows a consistent pipeline:

$$\textit{Task + Robot Capability + World State + Skills} \rightarrow \textit{LLM Inference} \rightarrow \textit{Generated Plan} \rightarrow \textit{Evaluation}$$

Using identical representations and evaluation criteria across models allows the experiments to measure differences in planning performance while controlling the surrounding framework.

The experimental design therefore evaluates three important dimensions of the proposed approach: **cross-task performance, cross-environment generalization, and adaptation to different robot capabilities**. The resulting evaluation scores can additionally serve as a baseline benchmark for comparing model selection and subsequent parameter-efficient fine-tuning (PEFT) [9] experiments.

9 Evaluation Methodology

Evaluating an LLM-generated robotic plan requires more than measuring textual similarity, since multiple action sequences may correctly accomplish the same task. Therefore, this dissertation uses a **deterministic software-based evaluation module** rather than another LLM as the evaluator.

The evaluator validates the generated plan against the **task, robot capabilities, world state, skills library, and predefined constraints**. The major evaluation dimensions include:

- **Goal Achievement:** Whether the generated plan satisfies the intended task objective.
- **Action Validity:** Whether generated actions belong to the available skills and action space.
- **Object and Location Validity:** Whether referenced entities exist in the corresponding world state.
- **Capability and Constraint Validity:** Whether the selected robot possesses the capabilities required to perform the actions.
- **Logical Ordering:** Whether actions follow valid dependencies and preconditions.
- **Plan Feasibility:** Whether the complete sequence can reasonably be executed under the supplied robot and environmental constraints.

The evaluation pipeline is therefore:

$$\textit{Generated Plan} + \textit{Ground-Truth Context} \rightarrow \textit{Deterministic Validation} \rightarrow \textit{Metric Scores} \rightarrow \textit{Overall Performance}$$

Aggregating these metrics across robots, environments, tasks, and models enables systematic comparison of planning performance. This evaluation strategy also improves

repeatability and transparency, since identical plans evaluated under identical conditions produce consistent results without introducing the variability or bias of an LLM-based judge.

It is important to note that the evaluation framework is **entirely deterministic and grounding-based**. It checks the different constraints in the generated plan and verifies whether the plan is consistent with the supplied capability model, world state, and skills library. However, it is unable to judge whether the robot can actually complete the task or not in a physical or simulated environment. The evaluation metrics **only check the grounding, but not the logical completeness** of the plan. Actual task success depends on additional factors including low-level motion execution, real-world perception, actuator reliability, and environmental dynamics, none of which are within the scope of the evaluation module. This distinction is an important limitation of the current evaluation framework and motivates future integration with execution-level verification.

10 Implementation and System Integration

The proposed framework was implemented as a **modular Python-based pipeline**, allowing each component to be developed, tested, and modified independently. The implementation integrates robot capability extraction, structured world representations, a skills library, LLM inference, and deterministic plan evaluation.

The overall software workflow is:

$$\textit{Robot Description} \rightarrow \textit{Capability Extractor} \rightarrow \textit{Capability JSON}$$
$$\textit{World Definition} \rightarrow \textit{World State JSON}$$
$$\textit{Task} + \textit{Capabilities} + \textit{World State} + \textit{Skills} \rightarrow \textit{LLM} \rightarrow \textit{Structured Plan} \rightarrow \textit{Evaluator}$$

The implementation uses **ROS2-compatible robot descriptions** [6], Python, JSON-based representations, and open-source LLM inference frameworks. The capability extractor applies predefined rules to interpret robot structures and generate planning-relevant capabilities and constraints.

A common inference interface was developed so that different LLMs can receive the same structured inputs and produce plans using a consistent output schema. This supports controlled comparison between models without modifying the remaining system architecture.

The individual modules are integrated into an experimental pipeline for executing tasks across different robot-world combinations, storing generated plans, computing evaluation metrics, and comparing model performance. This modular implementation also enables future components, models, robots, and execution systems to be incorporated without redesigning the complete framework.

11 Experimental Results and Analysis

The experimental phase evaluates the framework across multiple **robots, environments, natural-language tasks, and LLMs**. Each model is provided with the same structured inputs and evaluated using the deterministic evaluation module, enabling a controlled comparison of planning performance.

The analysis focuses on:

- overall plan validity and goal achievement,
- capability and constraint compliance,
- action, object, and location validity,
- logical ordering of generated actions,
- performance variation across task complexity and environments,
- differences in planning behaviour across robot embodiments, and
- comparative performance of the selected LLMs.

The framework was evaluated across different **robots, environments, and natural-language tasks** using the deterministic evaluation module.

11.1 Overall Task Success

Across **270 evaluations**, **27.9%** achieved complete task success, while **72.1%** failed to fully satisfy the requested objective.

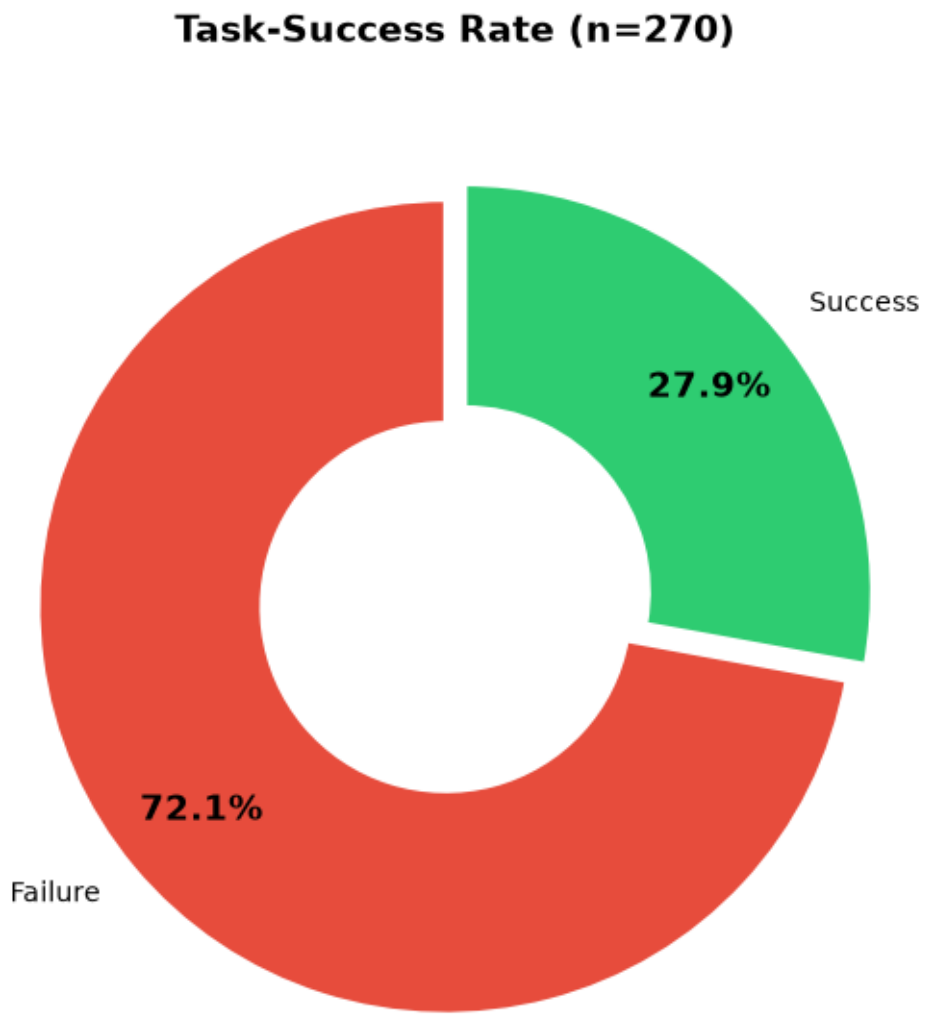


Figure 1: Overall task-success rate.

11.2 Metric-Wise Performance

Figure 2 shows that several individual planning metrics perform strongly despite the lower end-to-end task-success rate.

The framework achieves high PASS rates for **Sequence (99%)**, **Efficiency (97%)**,

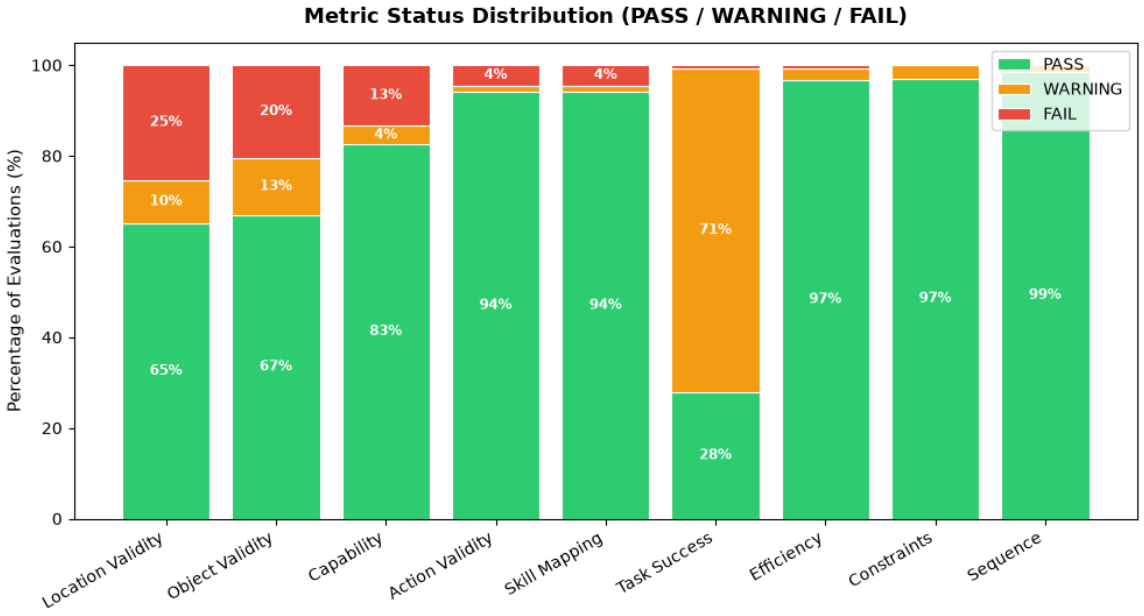


Figure 2: PASS, WARNING, and FAIL distribution across evaluation metrics.

Constraints (97%), **Action Validity (94%)**, and **Skill Mapping (94%)**. Capability validity reaches **83%**, while object and location validity show comparatively lower performance.

11.3 Robot and Environment Analysis

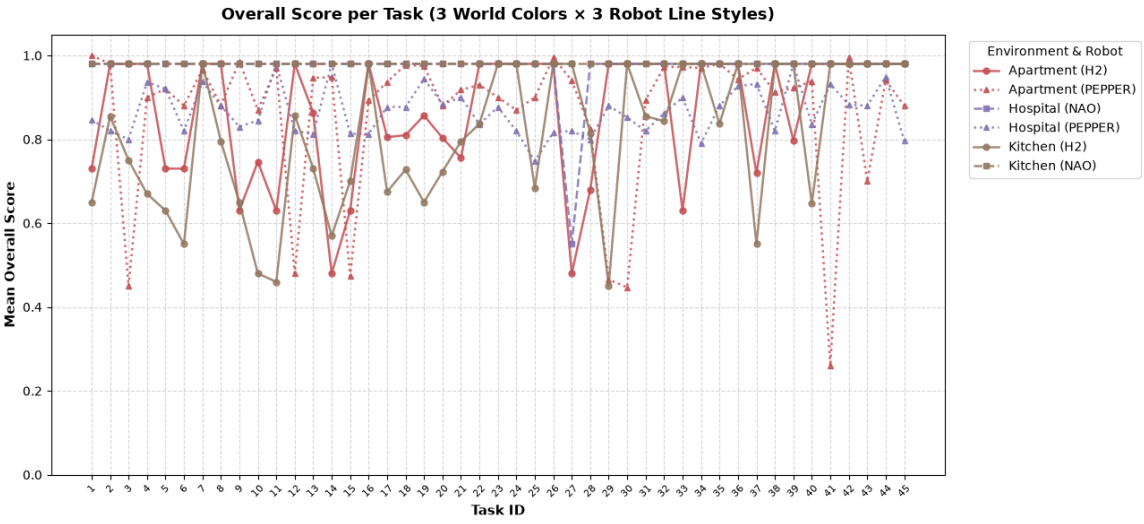


Figure 3: Task scores across robot and environment combinations.

Performance varies across tasks, robots, and environments, demonstrating that planning quality depends on both **robot capabilities and environmental context**.

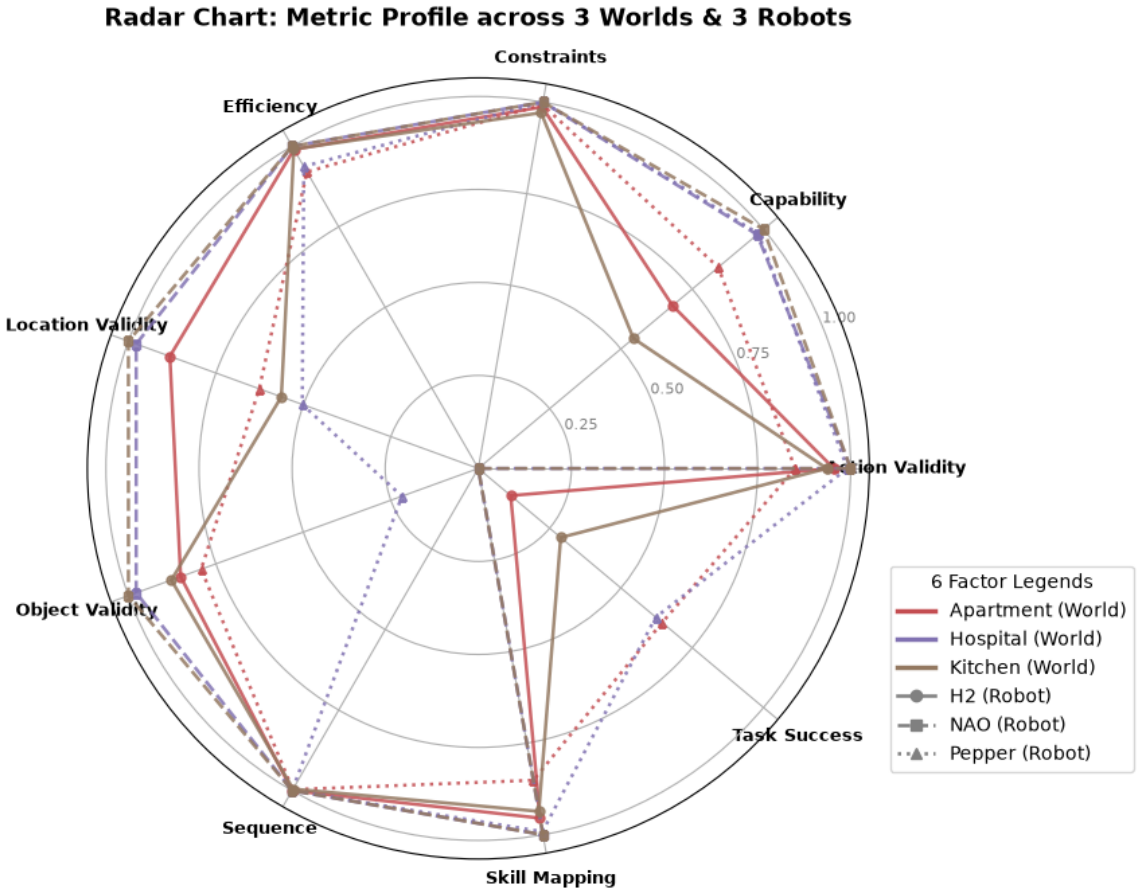


Figure 4: Metric profile across robot and environment configurations.

Overall, the results show that generated plans are frequently **structurally valid and constraint-compliant**, while complete task achievement remains more challenging. This establishes a useful baseline for model comparison and subsequent **PEFT/LoRA fine-tuning**.

The experiments are designed not only to identify the best-performing model, but also to understand **where and why planning failures occur**. Errors are categorized into cases such as unsupported actions, invalid environmental references, capability violations, incorrect action ordering, incomplete plans, and malformed structured outputs.

The resulting scores provide a quantitative baseline for the proposed framework. The strongest-performing model can subsequently be used for **Parameter-Efficient Fine-Tuning (PEFT)** [9], after which the same evaluation pipeline can be repeated to measure

whether domain-specific adaptation improves capability-aware robotic planning.

Note: Detailed quantitative results, comparative tables, graphs, and model-wise observations are to be incorporated in this section upon completion of the full inference and evaluation runs.

12 Limitations

Although the proposed framework establishes a structured approach for capability-aware robotic task planning, several limitations remain within the current scope of the dissertation.

- **Simulation and representation-based validation:** The generated plans are primarily evaluated using structured robot capabilities, world states, and software-based validation. Robot execution in simulation is not included in the current scope as it requires significant time for model finetuning, world building, and robot creation. This extensive setup will instead be a focus for future work.
- **Deterministic metrics check grounding, not logical completeness:** The evaluation framework checks grounding validity and constraint compliance deterministically, but it is unable to judge if the robot can actually complete the task or not. The metrics only check grounding, but not logical completeness. Actual task success depends on motion execution, real-world perception, and actuator reliability, which are outside the current scope.
- **Limited inference scale due to compute constraints:** The current experimental runs are limited in number due to available computational resources. A larger-scale evaluation across all robot-environment-task combinations requires significantly more compute capacity.
- **Zero-shot inference only:** The models are currently evaluated in a zero-shot setting. Fine-tuning on domain-specific data has not yet been performed, which

may limit the models' ability to consistently follow the structured output schema and capability constraints.

- **Static capability representation:** Robot capabilities are extracted from available descriptions and predefined rules. Dynamic factors such as hardware degradation, battery state, environmental conditions, and changing payload are not currently modeled.
- **Dependence on robot descriptions:** The accuracy of capability extraction depends on the completeness and correctness of URDF/Xacro [5] and associated robot information.
- **Predefined skills library:** The framework assumes a structured set of available robotic skills. Automatic discovery or learning of new skills is outside the present scope.
- **LLM limitations:** Generated plans can still contain hallucinated entities, unsupported actions, incorrect ordering, or incomplete reasoning despite capability and environment grounding.
- **High-level planning focus:** The dissertation primarily addresses task-level planning and does not attempt to replace low-level motion planning, trajectory generation, control, perception, or localization systems.
- **Benchmark scope:** The developed dataset provides controlled evaluation across multiple robots, environments, and tasks, but it cannot represent every real-world condition or robotic embodiment.

These limitations define the boundary of the current work rather than the final boundary of the proposed research direction. The framework provides a foundation upon which dynamic capability modeling, execution feedback, larger-scale benchmarking, and real-world robotic validation can subsequently be investigated.

13 Future Work

The current dissertation establishes a foundation for robot-agnostic and capability-aware task planning. Several directions can extend this framework into deeper research.

- **Fine-tuning and domain-specific model training:** Once a sufficient volume of capability-aware planning inferences and evaluation scores has been accumulated, the best-performing base LLM can be fine-tuned using parameter-efficient methods such as LoRA [9]. Alternatively, a dedicated lightweight neural network with a task-planning-specific architecture could be trained on the collected dataset, potentially replacing general-purpose LLMs for deployment in resource-constrained environments.
- **Multi-robot coordination for single tasks:** Extend the framework to support heterogeneous multi-robot teams collaborating on a single complex task. Different sub-tasks could be allocated to robots based on their complementary capabilities, with the LLM planner generating coordinated, role-aware action sequences that account for inter-robot dependencies and communication requirements. This extends beyond the current single-robot-per-task paradigm while reusing the same capability representation infrastructure.
- **Hierarchical and selective replanning (NavTopo-inspired):** Building on the checkpoint-based plan structure inspired by NavTopo [11], future work should implement full closed-loop replanning where only the affected checkpoint sub-task is regenerated upon failure, rather than regenerating the entire plan. This approach further reduces inference token cost and latency while preserving the already-validated portions of the plan.
- **Execution-level task success verification:** Integrate the planning framework with physical or simulation environments to measure actual task completion, moving beyond deterministic grounding checks. This would enable end-to-end evaluation from natural-language instruction to successful robot execution.

- **Real-world robot validation:** Deploy the framework on physical robots and compare software-based feasibility predictions with actual execution outcomes.
- **Dynamic capability modeling:** Extend static capability representations to account for battery state, payload, sensor availability, hardware failures, and changing environmental conditions.
- **Closed-loop planning:** Incorporate execution feedback so that plans can be dynamically corrected or regenerated when actions fail or the environment changes.
- **Task and Motion Planning integration:** Connect high-level LLM-generated task plans with motion planners [10] to verify geometric feasibility, reachability, collision constraints, and trajectory execution.
- **Automatic skill discovery:** Reduce dependence on manually defined skill libraries by discovering available skills and interfaces directly from robotic software ecosystems such as ROS2 [6].
- **Learning-based capability representation:** Investigate whether robot capabilities can be learned or refined from execution history instead of relying entirely on deterministic extraction rules.
- **Larger-scale benchmarking:** Expand the dataset to additional robots, environments, tasks, and language models to establish a broader benchmark for capability-aware robotic planning.

These directions provide a natural continuation toward doctoral research. The current framework can serve as the **base experimental platform**, while dedicated research can investigate adaptive capability representations, embodied feedback, cross-robot generalization, and ultimately the interaction between language reasoning and physical robot intelligence.

14 Conclusion

This dissertation presented a **robot-agnostic, capability-aware framework for language-conditioned robotic task planning**. The work addresses the gap between the semantic reasoning capabilities of Large Language Models and the physical and functional constraints of heterogeneous robotic systems.

The framework integrates **robot capability extraction, structured capability representation, world-state modeling, a common skills library, LLM-based planning, and deterministic plan evaluation**. By separating robot-specific information from the planning mechanism, the same architecture can be applied across different robots, environments, tasks, and language models.

A key outcome of this work is the principle that **a logically correct plan is not necessarily a robot-feasible plan**. Explicit representation of robot capabilities and environmental constraints is therefore essential when applying language models to robotic task planning.

The dissertation establishes a functional and extensible research foundation rather than attempting to solve general-purpose robotic planning completely. With dedicated research into real-world execution, adaptive capability learning, closed-loop planning, task-and-motion integration, and broader cross-robot evaluation, the proposed framework can be extended toward more general and reliable language-driven robotic intelligence.

15 Recommendations

Based on the design, implementation, and evaluation methodology developed in this dissertation, the following recommendations are proposed for further development of capability-aware robotic planning.

- **Capability information should be explicitly represented** rather than expecting an LLM to infer robot abilities from its pretrained knowledge.

- **Robot-specific information should remain separated from the planning model**, allowing the same planner to operate across different embodiments.
- **Structured world states and skill definitions should be used** to reduce hallucinated objects, locations, and unsupported robotic actions.
- **LLM-generated plans should be independently validated** before execution using deterministic capability, constraint, and logical checks.
- **Evaluation should extend beyond task success**, considering action validity, environmental grounding, capability compliance, logical ordering, and overall feasibility.
- **LLMs should function as high-level reasoning components**, while conventional robotics systems continue to handle perception, navigation, manipulation, motion planning, and control.

These recommendations support a modular approach in which advances in LLMs, robot hardware, capability extraction, and execution systems can be incorporated independently without redesigning the complete planning framework.

References

References

[1] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Ho, D., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jang, E., Ruano, R. J., Jeffrey, K., Jesmonth, S., Joshi, N. J., Julian, R., Kalashnikov, D., Kuang, Y., Lee, K.-H., Levine, S., Lu, Y., Luu, L., Parada, C., Pastor, P., Quiambao, M., Rao, K., Rettinghouse, J., Reyes, D., Sermanet, P., Sievers, N., Tan, C., Toshev, A., Vanhoucke, V., Xia, F., Xiao, T., Xu, P., Xu, S., and Zeng, A. "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances." *arXiv preprint arXiv:2204.01691*, 2022.

[2] Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., and Garg, A. "ProgPrompt: Generating Situated Robot Task Plans using Large Language Models." In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11523–11530, 2023.

[3] *[Full EMOS citation — please replace with complete author(s), title, venue, and year.]*

[4] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J., and Wen, J.-R. "A Survey of Large Language Models." *arXiv preprint arXiv:2303.18223*, 2023.

[5] Open Robotics. "URDF — Unified Robot Description Format." ROS Documentation, 2022. Available: https://wiki.ros.org/urdf [Accessed: July 2026].

[6] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W. "Robot Operating System 2: Design, Architecture, and Uses in the Wild." *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.

[7] Coleman, D., Sucan, I., Chitta, S., and Correll, N. "Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study." *arXiv preprint arXiv:1404.3785*, 2014. See also: MoveIt2 Documentation. Available: https://moveit.ros.org [Accessed: July 2026].

[8] Macenski, S., Martín, F., White, R., and Ginés Clavero, J. "The Marathon 2: A Navigation System." In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2718–2725, 2020.

[9] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. "LoRA: Low-Rank Adaptation of Large Language Models." In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[10] Garrett, C. R., Chitnis, R., Holladay, R., Kim, B., Silver, T., Kaelbling, L. P., and Lozano-Pérez, T. "Integrated Task and Motion Planning." *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, pp. 265–293, 2021.

[11] Gao, F., Peng, K., Zhang, H., Bermudez, I., and Payandeh, S. "NavTopo: Leveraging Topological Maps For Autonomous Navigation Of A Robot." *arXiv preprint arXiv:2410.01492*, 2024. See also: https://arxiv.org/abs/2410.01492 [Accessed: July 2026].

Glossary

Capability Extractor
: A software component that parses robot description files (URDF/Xacro) and derives structured capability representations for use in planning.

Capability JSON
: A JSON-formatted structured representation of a robot's capabilities, constraints, and operational parameters used as context input to the LLM planner.

Embodiment
: The physical and functional characteristics of a robot that determine its ability to perform tasks in a given environment.

Grounding
: The process of connecting high-level language or symbolic representations to concrete, executable actions and real-world entities.

Hallucination (LLM)
: The generation by an LLM of plausible-sounding but incorrect, non-existent, or unsupported objects, actions, or facts.

Large Language Model
: A neural language model pre-trained on large corpora that exhibits emergent capabilities for reasoning, planning, and language understanding.

PEFT
: Parameter-Efficient Fine-Tuning: a class of techniques for adapting pre-trained models to specific tasks by updating only a small subset of parameters, including methods such as LoRA.

Robot-agnostic
: Describing a system or framework designed to operate with any robot embodiment without requiring robot-specific modifications to the core architecture.

Skills Library
: A structured repository of abstract robotic skills with associated parameters, preconditions, and implementation mappings used to ground LLM-generated actions.

Task Planning
: The process of decomposing a high-level goal into a sequence of executable actions that achieve that goal.

URDF
: The Unified Robot Description Format, an XML-based file format used in ROS/ROS2 to describe the physical properties, kinematics, and dynamics of a robot.

World State
: A structured representation of the environment, including the locations, objects, properties, and relationships relevant to a given planning task.

Checklist of Items for the Final Dissertation Report

This checklist is attached as the last page of the final report and is to be duly completed, verified, and signed by the student.

No.	Description	Status
1	Is the final report neatly formatted with all the elements required for a technical report?	Yes
2	Is the Cover page in proper format as given in Annexure A?	Yes
3	Is the Title page (Inner cover page) in proper format?	Yes
4a	Is the Certificate from the Supervisor in proper format?	Yes
4b	Has it been signed by the Supervisor?	*[To sign]*
5	Is the Abstract included in the report properly written within one page?	Yes
6	Have the technical keywords been specified properly?	Yes
7	Is the title of your report appropriate? (Descriptive, precise, no uncommon abbreviations in title)	Yes
8	Have you included the List of Abbreviations/Acronyms?	Yes
9	Does the report contain a summary of the literature survey?	Yes
10	Does the Table of Contents include page numbers?	Yes
10i	Are the pages numbered properly? (Chapter 1 starts on page 1)	Yes
10ii	Are the figures numbered properly? (Figure numbers and titles at bottom of figures)	Yes
10iii	Are the tables numbered properly? (Table numbers and titles at top of tables)	Yes
10iv	Are the captions for figures and tables proper?	Yes
10v	Are the appendices numbered properly with appropriate titles?	Yes
11	Is the conclusion of the report based on discussion of the work?	Yes
12	Are references or bibliography given at the end of the report?	Yes
13	Have the references been cited properly inside the text of the report?	Yes
14	Are all the references cited in the body of the report?	Yes
15	Is the report format and content according to the guidelines?	Yes

Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.

Place: _____

Date: _____

Signature of the Student: _____

Name: **Sana Jaya Krishna**

ID No.: **2024AA05783**